Lock-Free Parallel SGD on Multicore Systems
CSC 258 Hogwild Project Report

Ethan Luvisia and Leo Liang

April 24, 2026

Abstract

This project studies a question that initially appeared simple, but ended up being pretty nuanced: when does Hogwild-style lock-free stochastic gradient descent actually help on a multicore CPU? Hogwild is attractive because it removes most lock overhead, but it also allows threads to read and write shared model parameters at almost the same time. The original intuition is that this is safe and fast when updates are sparse, because two randomly chosen examples usually touch different coordinates. Our implementation tested that intuition using a C++17/OpenMP benchmark framework, synthetic sparse logistic regression, and synthetic matrix factorization.

The main result is that Hogwild is useful, but not a perfect solution. On the short logistic scaling run, the serial baseline stayed fastest because each whole training job finished in only about 25 ms, so thread overhead dominated. On matrix factorization, where each update has more arithmetic work, parallel methods did produce real speedups; the best Hogwild scaling point was 12 threads at about 0.288 s compared with the serial runtime of about 0.460 s. The clearest Hogwild take-away came from the contention sweep: at low logistic contention, Hogwild beat striped locking, but when the sampled overlap probability rose to about 1.0, Hogwild became slower than striped locking. In other words, the project supports the Hogwild idea, but only under the conditions the idea actually assumes.

Contents

1 Introduction and Motivation

Stochastic gradient descent (SGD) is a common way to train models because each update only needs one sample, or a small batch of samples, instead of the whole dataset. This makes SGD a good candidate for parallelism: if we have many CPU cores, it is tempting to let many threads process training examples at the same time. The catch is that all of those threads usually want to update the same shared model vector. A straightforward implementation protects the model with locks, but locks can easily become the bottleneck. In the worst case, the program is "parallel" only in the sense that several threads are waiting for one shared lock.

Hogwild-style SGD attacks this problem by mostly removing locks. Threads compute gradients and directly update shared parameters. The reason this can work is sparsity. If each example only touches a few model coordinates, then most simultaneous updates will not overlap. Even when they do overlap, a small amount of noise or staleness may not damage convergence very much. That is the optimistic version of the story, and it is the one we wanted to test.

The concise systems question for our project was therefore:

> **When does Hogwild-style lock-free SGD beat synchronized alternatives on a real multicore machine, and when do synchronization, contention, and workload size reverse that advantage?**

We did not want the report to just say "Hogwild is faster" or "parallelism is better." We wanted to compare several concrete synchronization strategies under the same dataset seeds, model initialization, epoch budgets, and target-loss definitions. The project, in this way, therefore became both an implementation exercise and a benchmarking exercise. A big part of the work was making sure the graphs were comparing the synchronization strategies and not accidentally comparing different random datasets or different stopping rules.

2 Background: What Hogwild Is Trying to Exploit

The reference paper included in the repository, `hogwildTR.pdf`, frames Hogwild around sparse updates. In our project, we used the same basic idea but translated it into practical benchmark measurements. Each update has a footprint, which is the set of model coordinates it reads and writes. If those footprints are usually disjoint, then locking every update is wasteful. If the footprints overlap heavily, then lock-free execution can create more conflicting writes, more atomic retry pressure, and noisier optimization.

We tracked three simple dataset-level proxies in the CSV output:

- Ω: the approximate number of coordinates touched by one update. For logistic regression

this is the active feature count k. For matrix factorization it is $2 \times$ `rank`, because one update touches one user factor vector and one item factor vector.

- Δ: an empirical skew / incidence proxy. Higher values mean that some coordinates are touched much more often than average.
- ρ: a sampled overlap probability. This estimates how often two random updates share at least one coordinate, user, or item.

These are not exact theorem constants. They are useful measurement handles. For example, in the logistic contention sweep, ρ goes from around 0.002 in the low-contention case to about 1.0 in the most skewed case. That huge change is exactly what should stress Hogwild.

3 Implementation Overview

The codebase is a CPU-only benchmark framework written in C++17 with CMake. Parallel runs use OpenMP when the benchmark is built with an OpenMP-capable compiler. Python scripts run the experiment suites, collect CSV files, and generate the plots used in this report.

3.1 Algorithms Compared

We implemented five modes. The first four are used throughout the main scaling plots, and the fifth is a matrix-factorization stretch baseline.

- `serial`: one thread, plain SGD, and no synchronization.
- `coarse_lock`: parallel workers but one global mutex around each update. This is intentionally simple and conservative.
- `striped_lock`: the model is protected by multiple locks. Each update locks only the stripes that contain the coordinates it touches.
- `hogwild`: no mutex around an update. Each coordinate update is done with an atomic compare-and-swap style add, using relaxed memory ordering. This keeps the C++ implementation defined while still behaving like lock-free asynchronous SGD at the algorithm level.
- `local_batch_reduce`: used for matrix factorization only. Each thread accumulates local parameter changes and periodically merges them back into the shared model.

One small detail matters when reading the plots: at one thread, some synchronized algorithms effectively behave like serial code because there is no real inter-thread conflict to manage. The more interesting comparisons are at 2, 4, 8, and 12 threads.

3.2 Workloads

We used two synthetic workloads because they stress the system in different ways.

Sparse logistic regression. Each example has dimension d, but only k active coordinates. By changing k, model size, and hotspot skew, we can make updates sparse, dense, uniform, or highly contended. This workload is good for testing the pure sparse-update Hogwild idea. It is also very fast, which turned out to be important because thread-launch and scheduling costs can dominate small runs.

Matrix factorization. Each update uses one observed user-item rating and touches the factors for that user and that item. The footprint is sparse in the large parameter matrix, but each touched user/item vector has several rank coordinates. This gives more arithmetic per update than logistic regression and makes multicore execution more useful.

3.3 Learning-Rate Schedules

The project supports constant learning rate, epoch decay, and multiplicative backoff. Most main plots use backoff, with `backoff_gamma = 0.9`. We included the schedule sweep because convergence comparisons can be misleading if one method is helped by a different step-size policy. In the main experiments, the schedule was held fixed across algorithms within a workload.

4 Methodology and Fairness Controls

The most important part of the experiment design was making the comparisons fair. Each compared run used the same dataset generation settings, dataset seed, initialization seed, epoch budget, logging cadence, and target-loss definition. The full suite also used two trials and the scripts plotted medians. This does not remove all nondeterminism, especially for Hogwild interleavings, but it reduces the chance that a result is just caused by a lucky dataset or model initialization.

The main full-suite outputs are:

- `results_full/summary.csv`: one row per benchmark run, including runtime, throughput, target-loss time, final loss, and sparsity/contention proxies.
- `results_full/trace.csv`: epoch-level loss traces used for the convergence plots.

- `report/figures/`: copies of the generated plot images used by this LaTeX report.

The reported machine had 12 logical CPUs on macOS arm64. The `results_full/environment.txt` file records macOS 15.6, Python 3.11.5, and 12 CPU threads. The `build_gcc` CMake cache shows that the OpenMP build used `/opt/homebrew/bin/g++-15` with `-fopenmp`; the environment file separately records the default `c++` on the system as Apple clang. This distinction matters because OpenMP support is tied to the compiler used for the benchmark binary, not just the system default compiler.

4.1 Metrics

We used several metrics because no single number tells the whole story.

- **Runtime**: total wall-clock time for the run.
- **Throughput**: processed updates per second.
- **Speedup**: serial runtime divided by parallel runtime. Values above 1 mean a true wall-clock speedup.
- **Time-to-target-loss**: wall-clock time until the run first reaches a target loss taken from a reference serial run.
- **Final loss**: final optimization quality after the fixed epoch budget.

A run with very high throughput is not automatically the best if it reaches worse loss. In these experiments, however, most methods reached very similar final losses under matched budgets, so runtime and throughput are meaningful systems comparisons.

5 High-Level Results Before the Graphs

Before going graph by graph, the following list summarizes the most important numerical results from `results_full/summary.csv`. All values shown are medians across the full-suite trials where applicable.

- **Logistic thread scaling:** `serial@1` finished in 0.0251 s. The best Hogwild points were `hogwild@4` at 0.0580 s and `hogwild@8` at 0.0586 s, so the short logistic job did not have enough work for parallel overhead to pay off.
- **Matrix-factorization scaling:** `serial@1` took 0.4604 s. `hogwild@12` took 0.2882 s and `striped_lock@4` took 0.2495 s, so this workload produced real multicore speedups.

- **Logistic low contention:** at low overlap, `hogwild@12` took 0.1284 s while `striped_lock@12` took 0.1863 s. Hogwild beat the lock-striped version when conflicts were rare.
- **Logistic high contention:** at $\rho \approx 1.0$, `hogwild@12` took 0.2173 s while `striped_lock@12` took 0.1112 s. This is the clearest failure case for lock-free updates.
- **MF rank sweep:** at rank 64, serial took 0.6467 s and Hogwild took 0.4534 s. Larger rank increased arithmetic work enough that parallelism became more useful.

The short version of the list is: Hogwild works best when the update footprint is sparse enough and the run is large enough. It does not help much when the workload is tiny, and it can actively lose when almost every update collides with every other update.

6 Graph-by-Graph Results and Interpretation

This section includes every generated report figure. For each one, I describe what the graph is plotting, what pattern is visible, and what I think the pattern means.

6.1 Logistic: Time-to-Target vs Threads

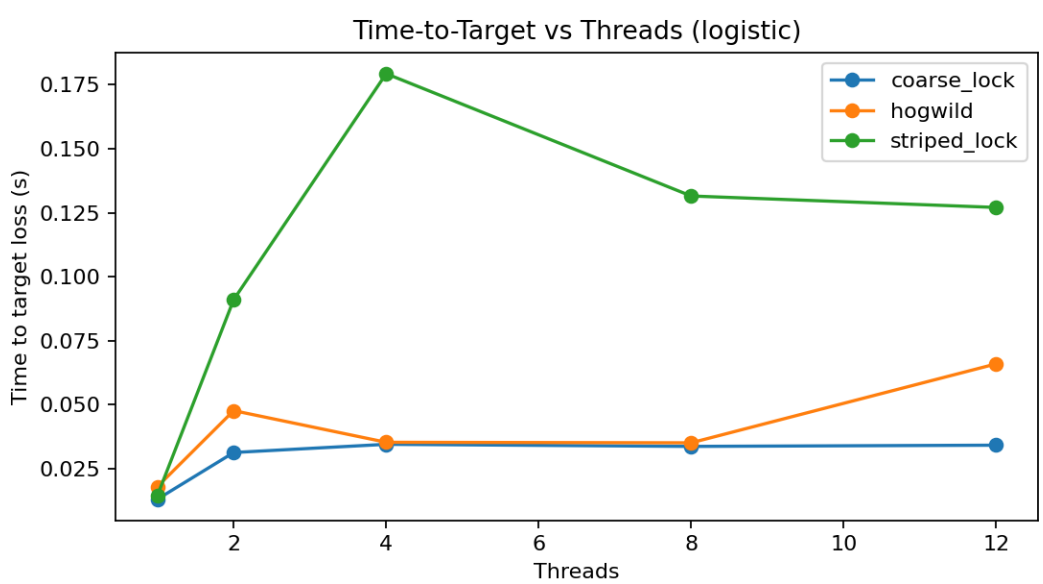

Figure 1: Logistic workload: time-to-target-loss vs thread count. Lower is better.

This graph measures how long each non-serial method needed to reach the reference target loss on the logistic scaling workload. The key pattern is that adding threads does not produce a clean downward curve. The best Hogwild points are around 4 to 8 threads, with median time-to-target

around 0.035 s, but the one-thread serial baseline reached the target in about 0.014 s. So even when Hogwild makes progress quickly, the run is too small for the parallel version to win end-to-end.

A useful way to read this graph is to separate optimization from overhead. The target loss is reachable by all methods, so the model is not failing to learn. The difference is mostly how much wall-clock overhead each method pays to get there. Coarse locking serializes much of the update work, striped locking pays many lock operations, and Hogwild pays atomic/CAS overhead plus OpenMP scheduling. On a larger logistic run, this overhead would be spread over more work. On this particular plot, it is a large fraction of the total runtime.

6.2 Logistic: Throughput vs Threads

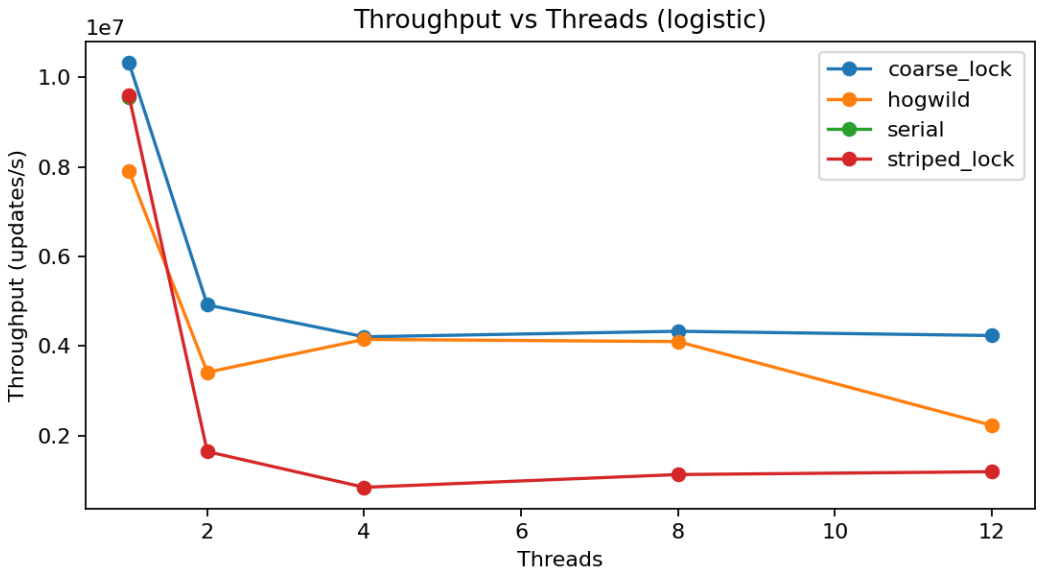

Figure 2: Logistic workload: update throughput vs thread count. Higher is better.

Throughput tells a slightly different story from time to target. The serial and one-thread paths process around 9-10 million updates per second. Once more threads are added, throughput drops for many parallel configurations instead of rising. This was one of the first signs that the logistic scaling configuration was too lightweight. Each update touches only 10 features, so there is not much computation to amortize the cost of scheduling work across threads.

Hogwild still compares reasonably against the heavier synchronized baselines. For example, around 4 to 8 threads it is far better than striped locking in this plot, because it avoids acquiring multiple locks per update. But the important lesson is that removing locks does not remove all overhead. Atomic coordinate updates, cache traffic, and thread coordination still matter. For this logistic case, those costs are big enough that multicore throughput does not scale the way we would hope.

6.3 Logistic: Speedup vs Threads

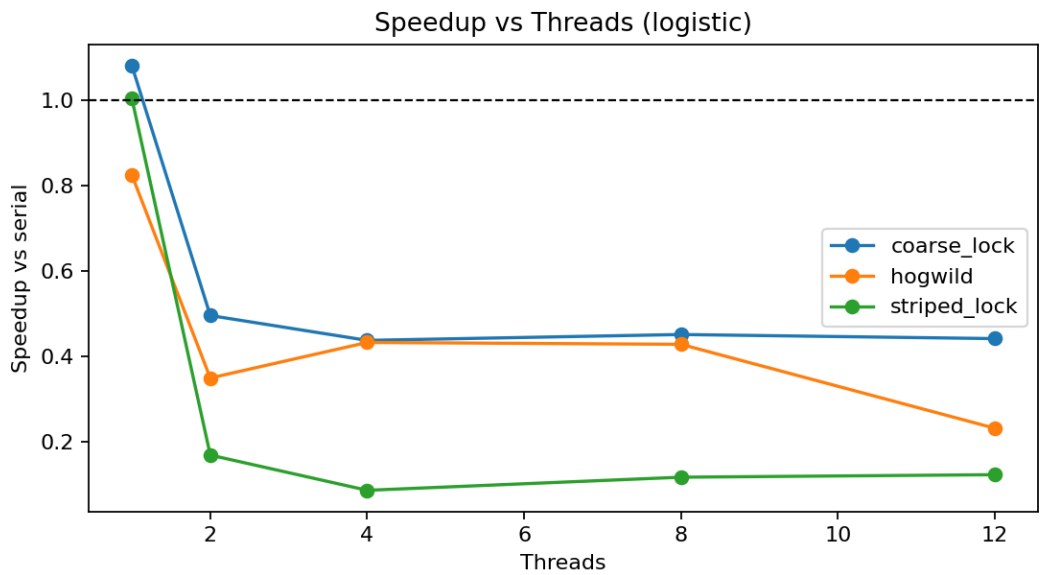

Figure 3: Logistic workload: speedup relative to the serial baseline. Values above 1 would mean parallel speedup.

This is the clearest visualization of the same logistic scaling issue. A speedup below 1 means the parallel version is slower than serial. Most logistic points stay below 1, and the best parallel points are still only around half of serial speed. That sounds bad for Hogwild, but it is really a warning about benchmark granularity. The total serial runtime is roughly 0.025 s, so there is very little room for a parallel implementation to recover its setup and synchronization costs.

This graph also shows why reporting only "updates per second" can be misleading. A method can look decent relative to another parallel method but still fail to beat the actual serial baseline. For the project question, this is important: Hogwild is not just competing against locks; it is also competing against the fact that simple serial code can be extremely fast for small sparse problems.

6.4 Logistic: Loss vs Time

This graph plots loss against elapsed time, so it combines algorithmic progress and systems overhead in one picture. The logistic loss starts near 0.694, which is close to the untrained binary classification loss, and drops to around 0.486 after the full epoch budget. The final losses across methods are very close, which means the methods are not diverging or learning very different models.

The separation between curves is therefore mostly about how fast each method can spend its update budget. Serial reaches the same loss values earlier in wall-clock time. Hogwild and coarse locking follow similar loss levels but at larger elapsed times, and striped locking tends to be the slowest

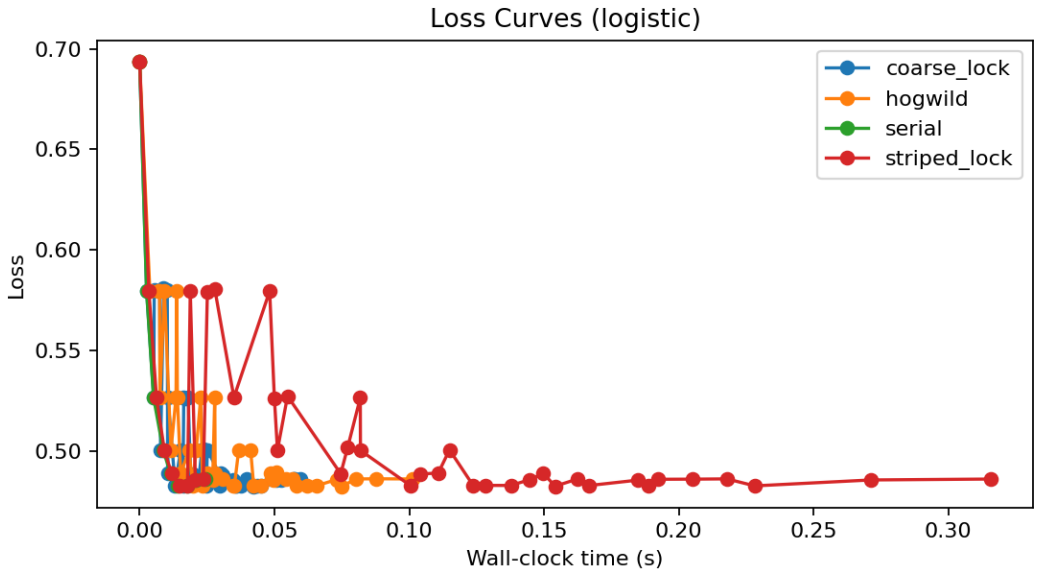

Figure 4: Logistic workload: convergence trajectory over wall-clock time.

because it repeatedly locks coordinate stripes. This graph made the project feel less like a pure machine-learning experiment and more like a systems experiment, since the optimization path is basically fine, but the machinery around the updates changes the wall-clock story.

6.5 Logistic: Loss vs Updates

This is one of the most useful verification plots. By putting updates on the x-axis, we remove most of the runtime effects. The curves almost lie on top of each other. That is exactly what we wanted to see, since for a fixed number of updates, Hogwild, locking, and serial SGD all reduce loss similarly on the low-contention logistic problem.

The practical takeaway is that the accuracy gap is not the main problem here. The main difference is how expensive each method is per update. If the loss-vs-updates curves had separated badly, then Hogwild would be changing the optimization behavior in a harmful way. Instead, the methods are optimization-equivalent enough that we can focus on systems-level costs such as locking, atomics, and cache contention.

6.6 Logistic: Performance vs Sparsity

This sweep changes k, the number of active features in each logistic update. Increasing k makes each update more expensive and also increases the chance that two examples share at least one coordinate. In the full data, ρ increases from about 0.00038 at $k = 4$ to about 0.55 at $k = 200$. So this is not just a compute sweep, it is also a conflict-probability sweep.

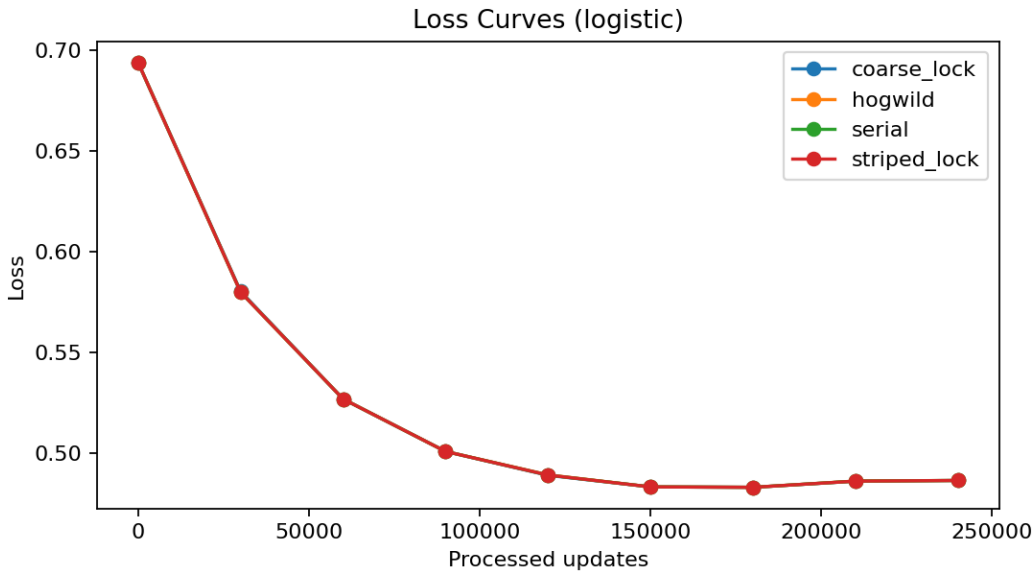

Figure 5: Logistic workload: convergence as a function of processed updates.

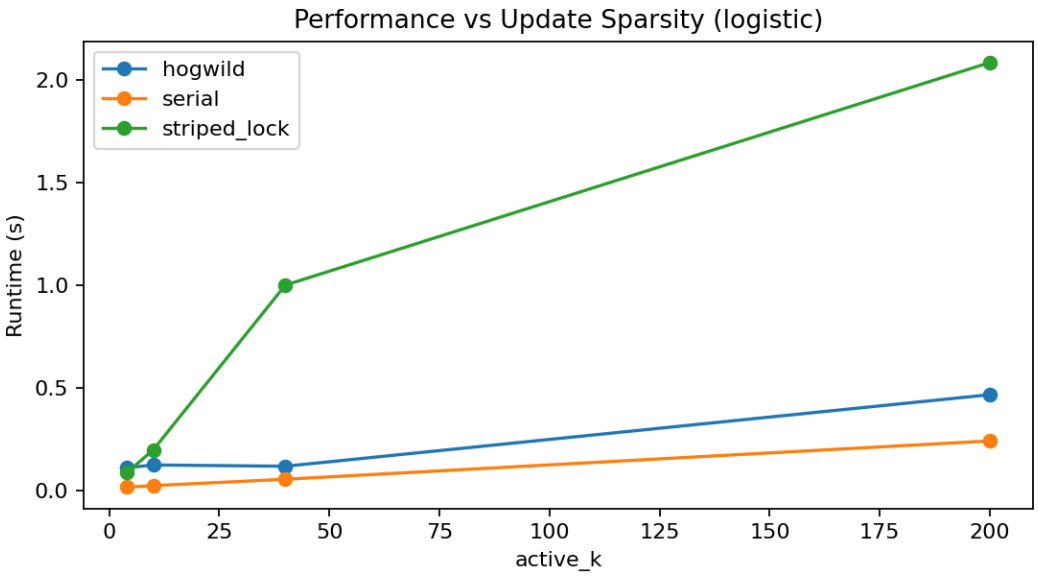

Figure 6: Logistic workload: runtime trend as active features per update, k, increases. Lower is better.

The graph shows that serial stays fastest for this logistic budget, but Hogwild is much better than striped locking once updates get larger. At $k = 40$, Hogwild takes about 0.118 s, while striped locking takes about 1.001 s. At $k = 200$, Hogwild takes about 0.467 s, while striped locking takes about 2.085 s. This is one of the strongest arguments for lock-free updates relative to fine-grained locks: acquiring and releasing many coordinate locks becomes very expensive as the update footprint grows.

One subtle point is that Hogwild at $k = 4$ is not better than striped locking in the median numbers. That makes sense after thinking about it. With only four coordinates touched, the lock set is tiny, so striped locking is not very expensive. As k grows, the lock-management cost grows too, and Hogwild starts to look much better relative to that synchronized baseline.

6.7 Logistic: Performance vs Contention

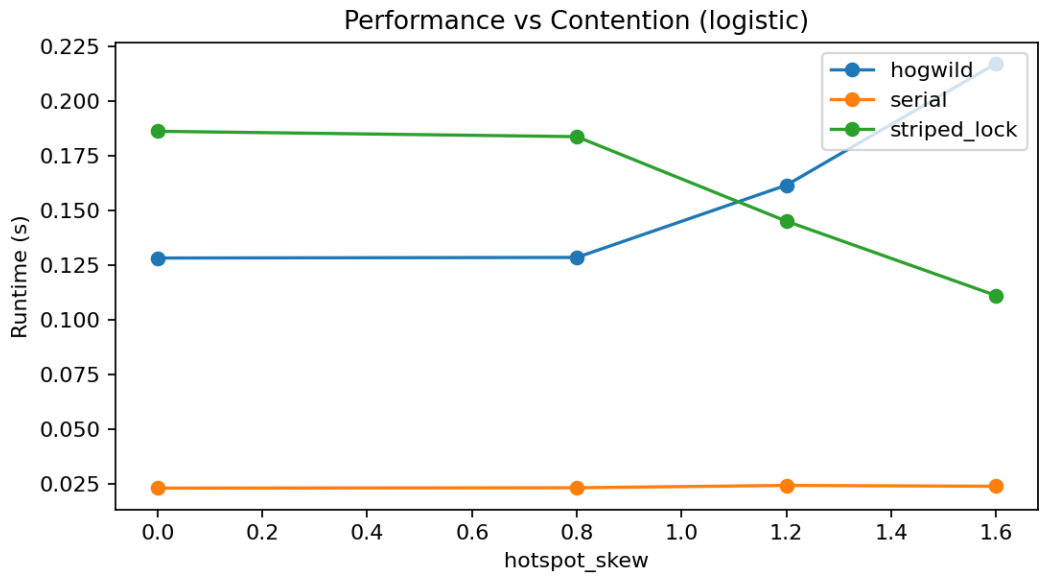

Figure 7: Logistic workload: runtime under increasing hotspot skew/contention. Lower is better.

This is probably the most important graph in the report. The x-axis changes hotspot skew, which makes the data distribution less uniform. At skew 0.0, update overlap is very low: ρ is around 0.002. At skew 1.6, almost every sampled pair overlaps: ρ is about 1.0. This is close to the worst case for the Hogwild assumption.

At low contention, Hogwild beats striped locking. The median low-contention logistic runtime is about 0.128 s for Hogwild and 0.186 s for striped locking. That matches the theory, since if collisions are rare, locks mainly add overhead. At extreme contention, the ranking flips. Hogwild takes about 0.217 s, while striped locking takes about 0.111 s. In other words, once the data concentrates on the same coordinates, allowing everyone to update lock-free is no longer the best systems choice.

This graph changed how I would explain Hogwild. It is not a blanket replacement for synchronization. It is a bet that update conflicts are rare enough that the cost of occasional stale/conflicting writes is lower than the cost of locking every update. When the conflict probability gets close to one, that bet loses.

6.8 Logistic: Performance vs Model Size

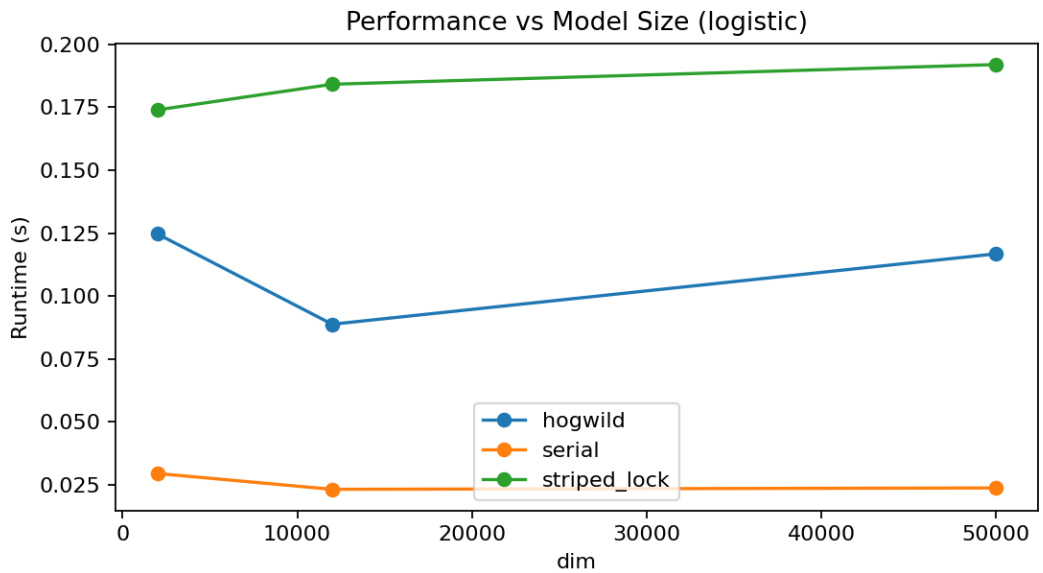

Figure 8: Logistic workload: runtime versus model dimensionality d. Lower is better.

The model-size sweep changes d while keeping the active feature count small. Intuitively, a larger model should reduce collision probability because active coordinates are spread over a larger space. The data reflects that: the sampled overlap ρ is around 0.0469 at $d = 2000$, around 0.0082 at $d = 12000$, and around 0.0020 at $d = 50000$.

Hogwild is consistently faster than striped locking in this sweep. At $d = 12000$, for example, Hogwild is about 0.0888 s while striped locking is about 0.1841 s. Still, serial remains around 0.023-0.030 s across the sweep. So model size helps reduce conflicts, but it does not by itself guarantee that parallelism wins. There still needs to be enough computation per run to make using many threads worthwhile.

This graph is a good reminder that sparsity has two dimensions, update size and parameter-space size. A small k is helpful, but a large d is also helpful because it spreads the updates out. Hogwild likes the combination of small update footprint and large model space.

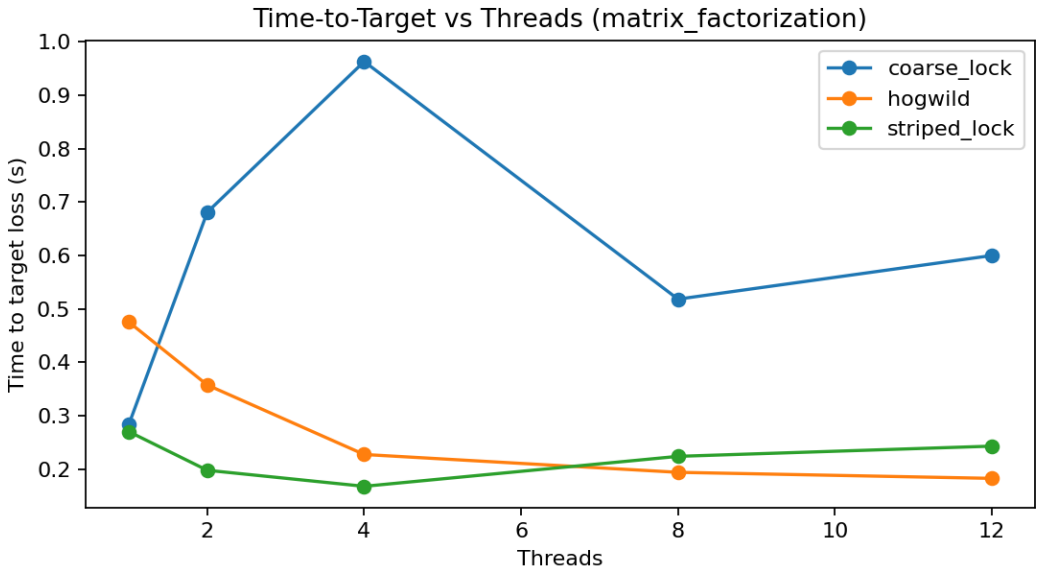

Figure 9: Matrix factorization workload: time-to-target-loss vs thread count. Lower is better.

6.9 Matrix Factorization: Time-to-Target vs Threads

Matrix factorization behaves differently from the short logistic scaling run. Each update has more arithmetic because it computes and updates rank-dimensional user and item vectors. This means the system has more useful work to do per scheduling decision, and parallelism has a better chance to help.

The time-to-target graph shows that both striped locking and Hogwild can reach the target faster than serial in some thread configurations. In the median scaling data, serial reaches the target in about 0.309 s. Hogwild at 12 threads reaches it in about 0.182 s, and striped locking at 4 threads reaches it in about 0.168 s. So unlike the logistic scaling plot, this is a case where multicore training improves the actual end-to-end learning time, not just a raw throughput number.

It is also worth noticing that the best point is not necessarily the highest thread count for every algorithm. Striped locking peaks around 4 threads in the scaling data and then gets slower. That suggests extra threads eventually increase lock/cache pressure more than they increase useful work.

6.10 Matrix Factorization: Throughput vs Threads

This throughput graph shows a healthier scaling pattern than the logistic one. Serial matrix factorization processes about 4.57 million updates per second. Hogwild reaches about 7.29 million updates per second at 12 threads, and striped locking reaches about 8.42 million updates per second at 4 threads. These are not linear speedups, but they are real improvements.

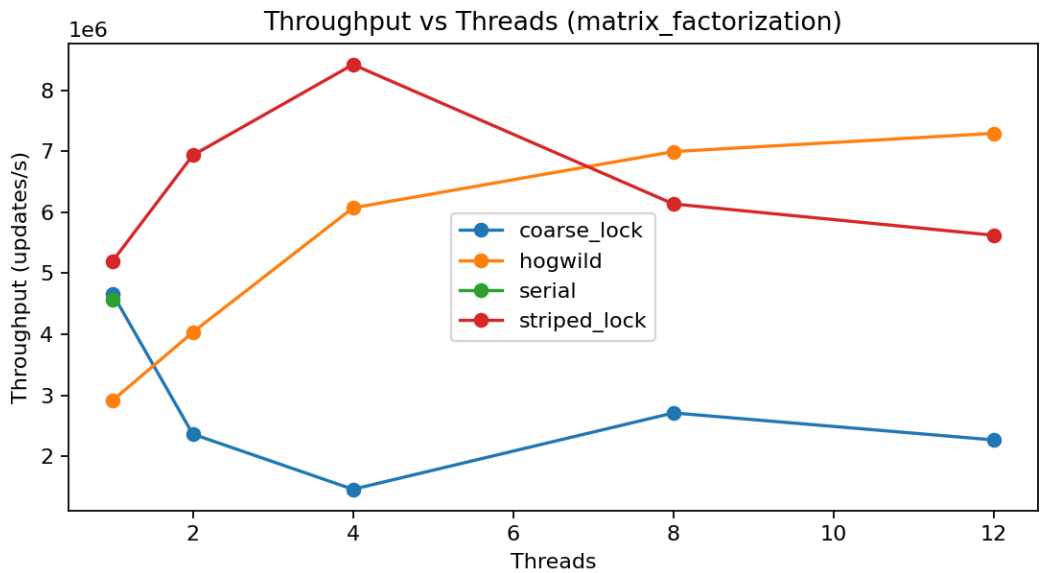

Figure 10: Matrix factorization workload: update throughput vs thread count. Higher is better.

The reason is mostly workload granularity. A matrix-factorization update touches $2 \times$ `rank` coordinates and does a dot product plus vector updates. In the main run, the rank is 32, so each update touches 64 factor coordinates. There is enough arithmetic that the fixed costs of parallelism are less dominant. The result is closer to what we hoped to see from Hogwild at the beginning of the project.

At the same time, the graph is still not a perfect upward line. Hogwild improves as threads increase, but with diminishing returns. Striped locking improves at first and then loses performance at higher thread counts. That is the systems tradeoff in one picture: more workers help until memory traffic, lock contention, and coordination start eating the gains.

6.11 Matrix Factorization: Speedup vs Threads

This speedup plot is the strongest positive result for multicore execution. Several points are above 1, meaning they beat the serial baseline in wall-clock time. The best striped-lock point is around 1.85x speedup at 4 threads. Hogwild reaches about 1.60x speedup at 12 threads.

The plot also shows why the answer is not simply "Hogwild always wins." In this particular MF scaling run, striped locking has the best single point. That does not contradict the Hogwild idea, because matrix factorization can have repeated users/items and therefore real contention. If a moderate amount of locking prevents some harmful overlap without adding too much overhead, striped locking can be competitive. The better conclusion is that Hogwild is one strong point in the design space, not the only good design.

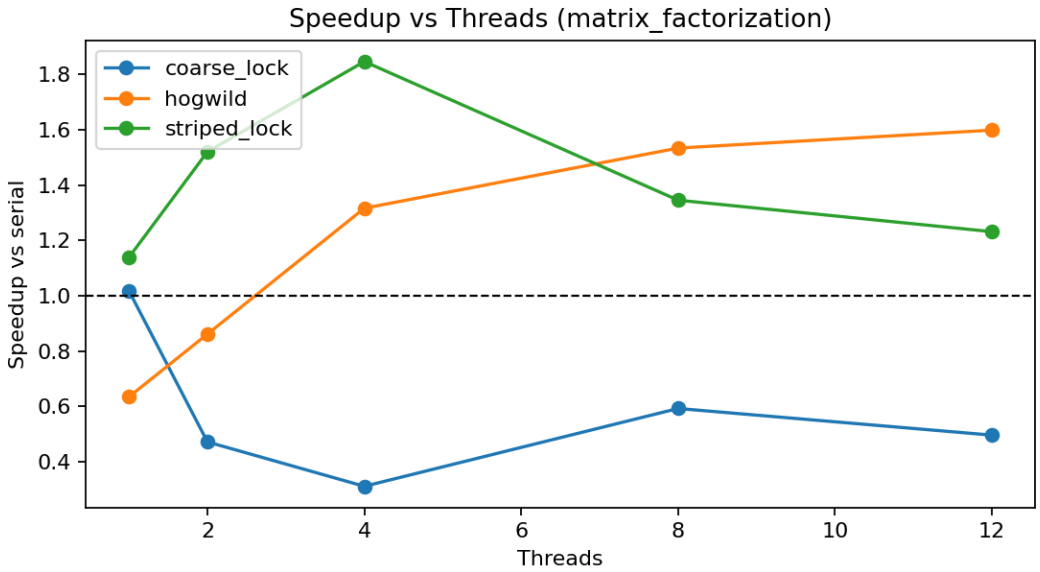

Figure 11: Matrix factorization workload: speedup relative to the serial baseline. Values above 1 mean true speedup.

6.12 Matrix Factorization: Loss vs Time

The MF loss curves show that the parallel methods are not just going faster by doing worse optimization. The loss starts around 0.092 and drops toward about 0.084 under the fixed epoch budget. The curves reach similar losses, but the wall-clock times differ. That is exactly the result we want from a parallel SGD implementation: similar optimization behavior with less elapsed time.

In the trace data, the median Hogwild curve reaches epoch 6 in roughly 0.375 s for the representative scaling traces, while serial reaches the same epoch in roughly 0.476 s. Striped locking is also fast in this workload, reaching the same point in roughly 0.337 s in the representative trace. The exact ranking varies by metric, but the broader conclusion is stable: matrix factorization has enough per-update work for parallelism to matter.

7 Additional Findings from the CSV Sweeps

Not every sweep has a separate figure in the final plot set, but the CSV contains useful extra information.

7.1 Schedule Sweep

The logistic schedule sweep compared constant learning rate, epoch decay, and multiplicative back-off for serial and Hogwild. All three schedules ended with similar losses in this configuration.

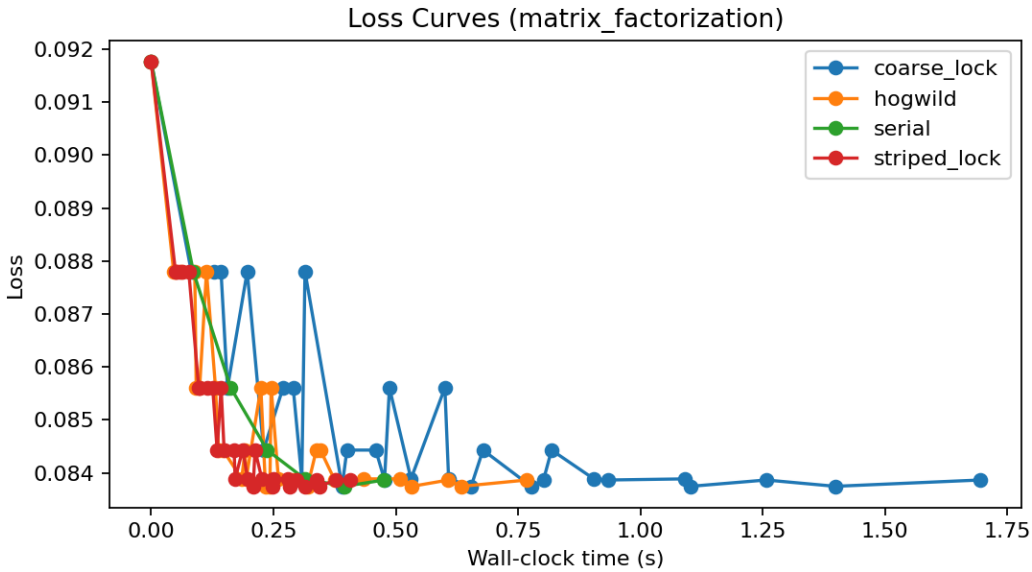

Figure 12: Matrix factorization workload: convergence trajectories over wall-clock time.

Constant learning rate had the lowest final loss in the median rows, around 0.4834, but it was not dramatically different from epoch decay or backoff. For Hogwild, the schedule choice changed time-to-target by a few hundredths of a second, but the synchronization effects were still larger than the schedule effects.

The main lesson is that the project results are not based on a fragile learning-rate setting where only one algorithm happens to work. The rankings we cared about were mostly systems rankings.

7.2 MF Contention Sweep

The matrix-factorization contention sweep showed a version of the same pattern as logistic contention, but not exactly the same ranking. At skew 0.0, striped locking was slightly faster than Hogwild: about 0.269 s versus 0.290 s. At skew 0.8, Hogwild was the fastest of the three main methods: about 0.291 s versus 0.382 s for striped locking and 0.403 s for serial. At skew 1.4, both parallel methods became much slower than serial, with Hogwild around 1.180 s and striped locking around 1.609 s.

This tells me that contention has workload-specific effects. In matrix factorization, the work per update is heavier, so Hogwild can still be useful at moderate skew. But at extreme skew, hot users/items concentrate too much of the traffic, and both synchronization strategies suffer.

7.3 MF Rank Sweep and Local Batch Reduce

The MF rank sweep is a good example of why per-update work matters. At rank 8, Hogwild was slightly faster than serial: 0.197 s versus 0.211 s. At rank 64, Hogwild was much more clearly faster: 0.453 s versus 0.647 s. Increasing rank increases the update footprint Ω, but it also increases arithmetic intensity, so the CPU has more useful work to parallelize.

The local-batch-reduce baseline did not perform well in this implementation. At rank 64 it took about 10.788 s, much slower than serial, striped locking, or Hogwild. The reason is probably the merge step, where each thread accumulates changes locally and then has to combine a large parameter vector back into the shared model. That reduces fine-grained locking, but the full-vector reductions are too expensive for this setup.

8 Reflection: What We Learned

The biggest thing we learned is that benchmarking parallel code is less obvious than writing the parallel code. At the beginning, we expected Hogwild to win most sparse cases as soon as there were several threads. The actual results were more conditional. If the workload is only a few milliseconds or a few tens of milliseconds, the overhead of OpenMP scheduling, atomics, and cache movement can be larger than the savings from removing locks. This is why the logistic scaling plot looks disappointing even though the algorithm is sparse.

We also learned that there are two different comparisons that can get mixed together. One comparison is Hogwild versus lock-based parallel SGD. Hogwild often looks good there, especially when striped locking has to acquire many locks per update. The second comparison is Hogwild versus serial SGD. That is harder. Serial code has no synchronization overhead at all, so Hogwild only wins when there is enough total work to amortize parallel overhead.

The contention sweep was the most satisfying result because it directly matched the theory. We could make the data more uniform and see Hogwild beat striped locking, then make the data extremely skewed and watch that advantage disappear. That made the abstract idea of "sparse conflicts" much more concrete.

Another practical lesson is that final loss alone is not enough. Several methods reached almost identical losses, but with very different runtimes. The loss-vs-updates graphs were helpful because they showed that the optimization behavior was similar. After that, the runtime graphs could be interpreted as systems overhead rather than as evidence that one method was learning a completely different model.

Finally, the project made us appreciate how many details go into a fair experiment: using the same initialization, checking target losses, making sure plots use medians, recording environment

information, and keeping the CSV schema stable. Without those details, it would be very easy to overstate a speedup that was actually caused by a different random seed or an easier target.

9 Limitations

There are several limitations that are worth being honest about.

- **Small runtimes are noisy.** Some logistic runs are around 20–30 ms, where measurement noise and fixed overheads can matter a lot.
- **No thread pinning.** The experiments do not pin worker threads to specific cores, so OS scheduling can affect cache locality and run-to-run variation.
- **Synthetic data.** Synthetic workloads are useful for controlled sweeps, but real sparse datasets may have different feature distributions and memory behavior.
- **Contention proxies are empirical.** Ω, Δ, and ρ help interpret the runs, but they are not a full theoretical model of convergence.
- **Atomic Hogwild is safer but not free.** Using atomic/CAS coordinate adds avoids undefined C++ data races, but it may cost more than a lower-level unsafe Hogwild implementation would cost.
- **Epoch-level logging is coarse.** Loss is recorded at epoch boundaries, so very short-term convergence differences may not appear in the trace plots.

These limitations do not invalidate the main trends, but they do mean that the exact crossover points should not be treated as universal constants. On a different CPU, compiler, memory system, or real dataset, the best thread count could change.

10 Future Work

A few extensions would make the project stronger. First, it would be useful to add thread affinity controls and run the suite on a Linux machine with performance counters. That would make it easier to separate lock waiting, cache misses, and atomic retry costs. Second, the project could include real sparse datasets in addition to synthetic ones. Third, we could implement a hybrid method that locks or serializes only known hotspot coordinates while leaving the rest of the model Hogwild. The contention sweep suggests that this kind of hybrid could be useful.

I would also add more plotting around the MF contention and rank sweeps. The CSV already has those results, but the current figure set focuses more on logistic sweeps and main thread scaling. A few extra figures would make the final story even clearer.

11 Conclusion

The final answer to the project question is conditional. Hogwild-style lock-free SGD can beat synchronized alternatives when updates are sparse, contention is low or moderate, and the workload has enough arithmetic work to amortize thread overhead. It is not automatically faster than serial SGD, and it is not automatically better than striped locking under high contention.

The most important evidence is the contrast between three cases. In the short logistic scaling run, serial remained fastest because the whole job was tiny. In matrix factorization, Hogwild reached real speedups because each update had more work. In the logistic contention sweep, Hogwild beat striped locking at low overlap but lost badly when ρ approached 1.0. Together, these results match the main Hogwild intuition while also showing the practical systems caveats.

So the project did not end with a slogan like "locks are bad" or "Hogwild always wins." The better conclusion is: **match the synchronization strategy to the workload**. For sparse, low-conflict, sufficiently large training jobs, Hogwild is a strong design. For tiny jobs or hotspot-heavy data, the safer synchronized methods, or even plain serial SGD, can be the better choice.